

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Бэк-энд разработка

Отчет

Лабораторная работа №3

Выполнил:

Рыбинская Злата

Группа

K3344

Проверил:

Добряков Д. И.

Санкт-Петербург

2026 г.

1 Постановка задачи

Реализовать Dockerfile для каждого сервиса; написать общий docker-compose.yml; настроить сетевое взаимодействие между сервисами.

2 Ход работы

Лабораторная работа посвящена контейнеризации многосервисного приложения с использованием Docker. Цель работы состоит в создании изолированной среды для запуска каждого сервиса, объединении сервисов в единую систему и настройке их взаимодействия через сеть.

В рамках работы рассматривалось приложение, состоящее из трех микросервисов: users-service, recipes-service и interactions-service. Каждый сервис реализован на базе NestJS и представляет собой отдельное серверное приложение с собственной бизнес-логикой и API. Для обеспечения обмена сообщениями между сервисами используется брокер сообщений RabbitMQ.

На первом этапе для каждого сервиса был создан Dockerfile. В Dockerfile задается базовый образ node:20-alpine, который содержит необходимое окружение для запуска Node.js приложений. Далее определяется рабочая директория внутри контейнера, копируются файлы зависимостей package.json и package-lock.json, после чего выполняется установка зависимостей с помощью npm install. Затем в контейнер копируется весь исходный код сервиса и выполняется сборка проекта командой npm run build. В завершение указывается порт, на котором работает сервис, и команда запуска приложения через node dist/main.js. Такой подход позволяет получить изолированный контейнер для каждого сервиса с полностью готовым к запуску приложением.

На втором этапе был создан общий файл docker-compose.yml, который описывает все сервисы системы и их конфигурацию. В данном файле были объявлены четыре контейнера: три контейнера для микросервисов и один контейнер для RabbitMQ. Для RabbitMQ использовался официальный образ

rabbitmq:3-management, который дополнительно предоставляет веб-интерфейс для управления брокером сообщений. Для каждого сервиса указана директория сборки, проброшены порты между хост-машиной и контейнером, а также заданы переменные окружения, включая порт приложения и адрес брокера сообщений. Все сервисы были объединены в одну пользовательскую сеть типа bridge, что обеспечивает возможность обращения к контейнерам по их именам.

Особое внимание было уделено настройке сетевого взаимодействия. В исходном коде сервисов адрес подключения к RabbitMQ был изменен с localhost на имя контейнера rabbitmq. Это необходимо, так как внутри Docker каждый контейнер имеет собственное сетевое пространство, и localhost указывает только на сам контейнер. Использование имени сервиса позволяет контейнерам корректно находить друг друга внутри общей сети. Аналогично при необходимости HTTP-взаимодействия между сервисами используется обращение по именам контейнеров, например http://users-service:3001.

Дополнительно была выполнена настройка запуска приложений внутри контейнера. В коде NestJS приложений серверы были сконфигурированы на прослушивание адреса 0.0.0.0, что позволяет принимать внешние подключения из Docker-сети и с хост-машины. Также были заданы переменные окружения для портов и адреса RabbitMQ, что повышает гибкость конфигурации и упрощает перенос приложения.

После завершения настройки была выполнена сборка и запуск системы с помощью команды docker-compose up --build. В результате были успешно запущены все контейнеры, обеспечено их взаимодействие через RabbitMQ и проверена доступность API каждого сервиса через проброшенные порты. Дополнительно была проверена работа веб-интерфейса RabbitMQ, доступного на порту 15672.

3 Вывод

В ходе выполнения лабораторной работы были получены практические навыки контейнеризации приложений, создания Dockerfile и настройки docker-compose. Было показано, как организовать взаимодействие между несколькими сервисами в изолированной среде и как использовать брокер сообщений для асинхронного обмена данными.

В результате работы создана полностью функционирующая многосервисная система, запускаемая одной командой. Контейнеризация упростила процесс развертывания и обеспечила воспроизводимость окружения. Это решение повышает удобство разработки, тестирования и дальнейшего масштабирования приложения.